

Evaluating CISPO Under Stale Rollouts in Asynchronous RL

Sathya Gnanakumar
sgnanaku@umd.edu

Ishaan Kalra
ishaank@umd.edu

Daniel Kiely
dmkiely@umd.edu

Parthav Poudel
ppoudel@umd.edu

Abstract

Asynchronous reinforcement learning (RL) systems improve throughput by decoupling rollout generation from policy updates. However, this design introduces stale rollouts, where data may be generated by an older policy and later consumed by a newer learner. Although recent work has proposed algorithms for off-policy and stale-data training, their comparative behavior in real systems such as AReaL remains understudied. We implement CISPO (Clipped IS-weight Policy Optimization) in AReaL and compare four RL algorithms (GRPO, CISPO, SAPO, and M2PO) under controlled staleness levels. On GSM8K with Qwen2.5-1.5B, CISPO achieves the highest final task reward at high staleness, reaching 0.8984 at $\eta = 4$ compared with 0.8281 for GRPO and 0.8594 for M2PO. Evaluation curves are less separated than training reward, suggesting that GSM8K may not fully expose generalization differences among the methods, but the training dynamics show that algorithm choice affects robustness to stale data. Our work provides an open-source CISPO implementation in AReaL and empirical evidence that CISPO is a strong candidate for moderate-to-high staleness asynchronous RL.¹

1 Introduction

Modern reinforcement learning (RL) systems for large language models (LLMs) face a fundamental trade-off between throughput and training stability. In a standard synchronous RL pipeline, post-training methods such as PPO [14] and GRPO [15] first generate batches of model outputs, or *rollouts*. A reward function then evaluates each rollout, and policy-gradient updates are applied to increase the probability of high-reward behavior [17]. Because the same policy generates and trains on the data, synchronous RL is relatively stable. However, this stability comes at a cost: training must wait for rollout generation to finish, leaving expensive GPU resources idle [10].

Asynchronous RL systems such as AReaL [3] address this inefficiency by decoupling rollout workers from training workers, enabling both to run continuously. This architectural change introduces *staleness*, the gap between when data is generated and when it is used for training. If model version v_i generates a rollout but a later model version trains on that data, the update is off-policy. Prior work has shown that such off-policy updates can destabilize RL post-training for LLMs [3, 18, 19].

Several optimizations of GRPO and PPO have been proposed to address staleness [18, 19], but it remains unclear which algorithms handle off-policy updates best under realistic workloads and system constraints. Prior work has often relied on simulation [19] or studied individual algorithms in isolation.

We study CISPO (Clipped IS-weight Policy Optimization) [1], an algorithm originally proposed for large-scale Mixture-of-Experts (MoE) reasoning models. CISPO clips importance-sampling weights while preserving token-level gradient signal, a design that may be useful when stale rollouts cause importance ratios to drift. AReaL is a widely used asynchronous RL framework with reference implementations of algorithms such as GRPO, SAPO, and M2PO [7]. Since AReaL did not include CISPO, we implement CISPO in the framework and compare it against these three baselines under controlled staleness. Our key contributions are:

- (1) The first implementation of CISPO in the open-source AReaL framework, enabling broader research adoption.
- (2) An empirical comparison of GRPO, CISPO, SAPO, and M2PO under controlled staleness levels in a real asynchronous RL system.

2 Related Work

Policy optimization methods form the foundation of modern reinforcement learning for language models. Proximal Policy Optimization (PPO) [14] established clipped surrogate objectives as a practical approach to stable policy updates, and became a standard component of reinforcement learning from human feedback (RLHF) pipelines for language model alignment [11, 20]. Recent work has focused on reinforcement learning from verifiable rewards (RLVR), designed to improve reasoning on tasks like math and coding. Shao et al. [15] introduced GRPO, which simplifies PPO by removing the critic model and has become the most widely used RL algorithm for post-training reasoning models.

Several recent works focus on improving throughput in RL training through asynchronous execution. Noukhovitch et al. [10] helped popularize asynchronous RL for language models and found online Direct Preference Optimization [13] to be the most robust to off-policy data among the RLHF algorithms they evaluated. They also found that asynchronous RLHF sped up training by 40–70%. However, this work focused on RLHF algorithms rather than RLVR algorithms for verifiable reasoning tasks.

Zheng et al. [19] introduce Second-Moment Trust Policy Optimization (M2PO), an algorithm that constrains the second moment of importance weights to avoid clipping gradients except in the presence of extreme outliers. They demonstrate that M2PO can handle data generated up to 256 updates off-policy, a regime more extreme than the staleness levels typically used in AReaL deployments [8]. Their study uses simulations that control the exact lag between rollout generation and policy updates. In systems such as AReaL, by contrast, rollout workers run continuously until batches are filled, so each batch may contain samples generated by multiple policy

¹Our code and experiment scripts are available at <https://github.com/danielkiely/AReaL>.

versions. Still, M2PO suggests that preserving gradients while controlling importance weights is central to robust asynchronous RL, motivating our study of CISPO.

Huang et al. [6] introduce Variance Control Policy Optimization (VCPO), a method that dynamically scales the learning rate to dampen unreliable updates. This method delivers a 2.5x speedup in training while matching synchronous performance in long-horizon, tool-use settings. However, VCPO requires per-trajectory gradient norm computation via a backend-specific single-backward-pass implementation in Megatron-LM [16]. This strategy is fundamentally different than clipping strategies such as M2PO, and we were unable to integrate this algorithm into AReaL due to the architectural assumptions made by the authors.

Piché et al. [12] and Li et al. [9] provide alternative asynchronous RL systems to AReaL, and we leave it to future work to explore how algorithms perform within these different systems. See Section 3.2 for an explanation of why we chose AReaL over other asynchronous RL systems.

3 Design

In this section we describe the AReaL framework that serves as our experimental platform and our rationale for selecting it.

3.1 The AReaL Framework

AReaL is an asynchronous reinforcement learning system designed for large-scale training of reasoning-oriented language models. Synchronous RL pipelines alternate between rollout generation and policy training, leaving GPUs idle whenever rollouts take longer than expected to complete. AReaL addresses this inefficiency by separating these phases into independently scheduled components that execute concurrently, reducing GPU idle time at the cost of off-policy updates. The framework consists of four main components, which together achieve up to a 2.77x training speedup compared to synchronous systems [3].

Interruptible Rollout Workers. A pool of inference workers continuously generates rollouts using the latest policy checkpoint available to them. The workers are interruptible: when the trainer publishes a new policy version, any in-flight generations can be preempted, their KV caches recomputed with the updated weights, and then resumed. This mechanism allows AReaL to bound staleness without forcing rollout workers to stall while waiting for training to complete.

Reward Service. A separate, CPU-bound service handles reward computation for completed rollouts. For a task such as GSM8K [2], this consists of parsing the model’s output and checking the numerical answer against the ground truth. Decoupling reward computation from the GPU prevents lightweight verification work from blocking rollout workers and simplifies the substitution of alternative reward functions.

Replay Buffer with Staleness Control. Completed (rollout, reward) tuples are aggregated into a buffer from which the trainer samples. The buffer enforces the staleness constraint η by tracking the policy version associated with each sample and rejecting tuples whose generating policy is more than η updates behind the current trainer. This constraint is exposed as a configurable parameter through which we control $\eta \in \{0, 2, 4\}$ in our experiments.

Trainer Workers and Parameter Service. Trainer workers consume batches from the buffer and apply policy updates using the configured loss function (GRPO, CISPO, SAPO, or M2PO in our experiments). Updated parameters are pushed to a parameter service that distributes them to the rollout workers asynchronously, closing the loop without imposing a synchronization barrier between training and rollout.

The resulting pipeline executes rollout and training in parallel, with η bounding how far the rollout workers can lag behind the trainer workers. Setting $\eta = 0$ recovers fully synchronous behavior, while larger values relax the constraint and permit rollout workers to run further ahead of the trainer. We note that even in traditional synchronous RL, mini-batch reuse means updates are rarely truly on-policy, as the policy shifts with each gradient step while the rollout data stays fixed. AReaL’s η parameter operates at a coarser granularity: it counts the number of full training batches by which a rollout lags behind the current policy, so $\eta = 0$ does not eliminate within-batch off-policyness, but rather ensures rollouts are never more than one batch stale.

3.2 Rationale for Choosing AReaL

We selected AReaL for three main reasons.

First, AReaL is actively maintained, has a substantial open-source community, and ships with reference implementations of GRPO, SAPO [4], and M2PO. This allowed us to focus our engineering effort on implementing CISPO rather than re-implementing each baseline from scratch, while also ensuring that our comparisons are tested and deployed implementations of each algorithm.

Second, it allows us to bound staleness using its η parameter, defined as the number of full training batches by which a rollout may lag behind the current policy. This gives us a direct way to compare algorithms at $\eta \in \{0, 2, 4\}$.

Third, AReaL provides well-defined hooks for plugging in custom loss functions without modifications to the scheduler, which kept our CISPO implementation localized rather than dispersed throughout the codebase.

4 Implementation

4.1 CISPO

GRPO [15] is the most widely used RL algorithm for reasoning post-training. It optimizes a clipped surrogate objective over token-level importance ratios:

$$\mathcal{L}_{\text{GRPO}} = -\mathbb{E} \left[\min \left(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (1)$$

where $r_t = \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$ is the token-level importance ratio and $\hat{A}_t$ is the advantage estimate. When r_t falls outside the trust region $[1 - \epsilon, 1 + \epsilon]$, the clip function takes over and the effective gradient contribution of that token is zeroed out.

CISPO [1] was proposed for synchronous training of large MoE reasoning models and modifies this clipping strategy. The motivation is that standard GRPO clipping zeroes out gradient contributions from rare reflective tokens such as “However” or “Wait”, which the authors argue are disproportionately important for chain-of-thought reasoning. CISPO instead clips the importance sampling

weight and uses it to scale the log-probability gradient directly:

$$\mathcal{L}_{\text{CISPO}} = -\mathbb{E}[\text{sg}(\text{clip}(r_t, 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}})) \cdot \log \pi_{\theta}(a_t | s_t) \cdot \hat{A}_t] \quad (2)$$

where $\text{sg}(\cdot)$ denotes a stop-gradient. Every token receives a gradient signal regardless of whether its ratio drifts outside the trust region, with out-of-range tokens receiving a scaled rather than zeroed gradient.

4.2 CISPO in Asynchronous RL

Although CISPO was not designed for asynchronous RL, its objective is naturally suited for stale-data training. As the behavior policy lags behind the current policy, importance ratios can drift away from one. In PPO-style objectives, clipping protects the update from extreme ratios, but it can also remove gradient signal from tokens whose ratios fall outside the trust region. Under staleness, this may reduce the amount of useful signal available for each update.

CISPO keeps every valid token in the objective while bounding the importance weight that scales its contribution. This suggests a useful compromise for asynchronous RL: stale samples remain part of the update, but their influence is capped. We therefore test whether this loss-level property improves robustness when rollout data is generated by older policy versions.

4.3 Implementation in AReaL

At the start of this project, no public CISPO implementation existed for AReaL despite the framework’s coverage of other algorithms. We added CISPO as a new loss function in the loss registry, with ϵ_{low} and ϵ_{high} exposed as configurable hyperparameters (`cispo_epsilon_low`, `cispo_epsilon_high`) following the naming convention of the existing clipping bounds. The implementation reuses AReaL’s importance ratio computation and integrates with the existing FSDP trainer backend without modification, so CISPO inherits the same checkpointing, gradient accumulation, and distributed training behavior as the other algorithms.

CISPO integrates naturally into AReaL’s loss dispatch logic alongside GRPO, SAPO, and M2PO: when `use_cispo_loss` is set, the trainer calls `cispo_loss_fn` in place of the default PPO objective. This design required no changes to AReaL’s rollout, reward, or buffer infrastructure, helping ensure that comparisons to the baseline algorithms isolate the loss function rather than scheduler behavior.

The implementation is released on our project fork and constitutes the main system contribution of this work, making CISPO available to the AReaL community. We intend to upstream our contribution to the main AReaL repository so that researchers will be able to use CISPO for their asynchronous training.

5 Experiments and Results

We evaluate whether CISPO’s loss-level behavior translates into improved robustness under asynchronous staleness. Our experiments keep the task, model, infrastructure, and most hyperparameters fixed while varying two factors: the RL algorithm and the allowed staleness level.

5.1 Experimental Setup

We evaluate four algorithms (GRPO [15], CISPO [1], SAPO [4], M2PO [19]) across three staleness levels ($\eta \in \{0, 2, 4\}$) on the GSM8K [2] mathematical reasoning benchmark using Qwen2.5-1.5B-Instruct. Training occurs on the Zaratán HPC cluster with 2 A100 GPUs per job for 10 epochs, totaling 12 experimental conditions.

We use consistent hyperparameters based on `gsm8k_grpo.yaml` configuration: batch size of 32, maximum 512 new tokens per generation, and evaluation every 20 training steps. We reduced batch size from AReaL’s default of 64 due to memory constraints. The staleness parameter η is controlled via `rollout.max_head_offpolicyness`. Algorithm-specific parameters are kept to a minimum: CISPO uses `cispo_epsilon_high=1.2`, and M2PO uses `m2_threshold=0.04`. Each job has a 12-hour time limit and typically completes in 8–10 hours.

5.2 Training Stability

Figure 1 summarizes final training rewards across all 12 experimental conditions, and Figure 2 shows the corresponding reward trajectories grouped by staleness level. At $\eta = 0$ (synchronous), all four algorithms perform competitively: CISPO achieves the highest final reward (0.9141), followed by M2PO (0.9062), GRPO (0.8828), and SAPO (0.8594). Although per-step reward is noisy, the synchronous setting shows that all four methods can learn the task under fresh data.

The algorithms diverge as staleness increases. GRPO degrades monotonically, falling from 0.8828 at $\eta = 0$ to 0.8359 at $\eta = 2$ and 0.8281 at $\eta = 4$, a total drop of 5.5 percentage points. This trend is consistent with GRPO’s token-level clipping reducing the learning signal as more stale tokens drift outside the trust region. CISPO is more robust over the same range: it drops from 0.9141 at $\eta = 0$ to 0.8750 at $\eta = 2$, but reaches 0.8984 at $\eta = 4$, the best final reward among all methods at high staleness. M2PO, despite being designed for off-policy robustness, declines from 0.9062 to 0.8984 and then 0.8594 as η increases. SAPO reaches 0.9219 at $\eta = 2$ but collapses to 0.0078 at $\eta = 4$ in our run. We discuss this failure mode further in Section 6.4.

The dense training curves in Figure 2 also show that final reward is not the only relevant signal. M2PO exhibits the smoothest reward trajectories, consistent with its second-moment trust-region motivation [19]. CISPO appears less variable than GRPO and SAPO at $\eta = 4$, but still exhibits substantial per-step noise. We therefore interpret CISPO’s advantage as a final-performance advantage under the tested staleness levels, not as evidence that it eliminates training variance.

5.3 Evaluation

Evaluation curves provide a complementary view of whether the training reward differences translate to held-out performance. Figure 3 shows that evaluation rewards are less separated than final training rewards: most runs remain in a narrow band despite visible differences in training reward. This suggests that GSM8K may be too easy, or the evaluation too coarse, to fully expose generalization differences among these algorithms over 10 epochs. In this

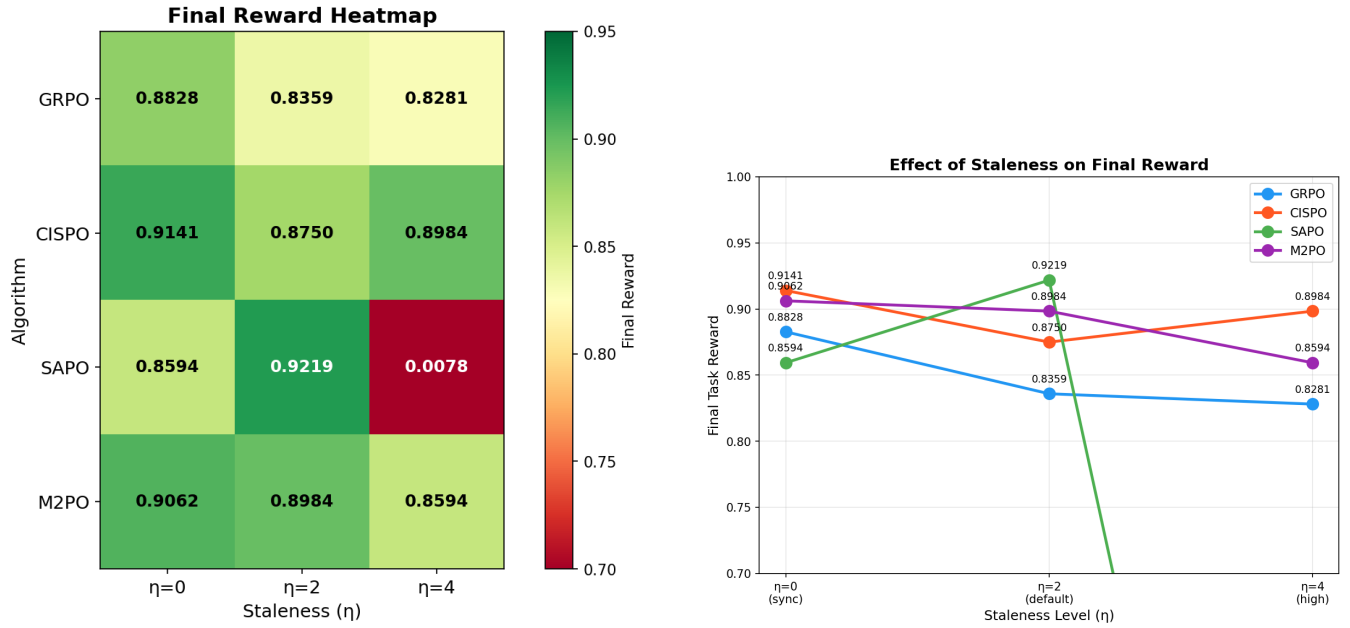


Figure 1: Left: Final training reward heatmap across algorithms and staleness levels. CISPO achieves the highest final reward at $\eta = 4$ (0.8984). Right: Final reward as a function of staleness for each algorithm.

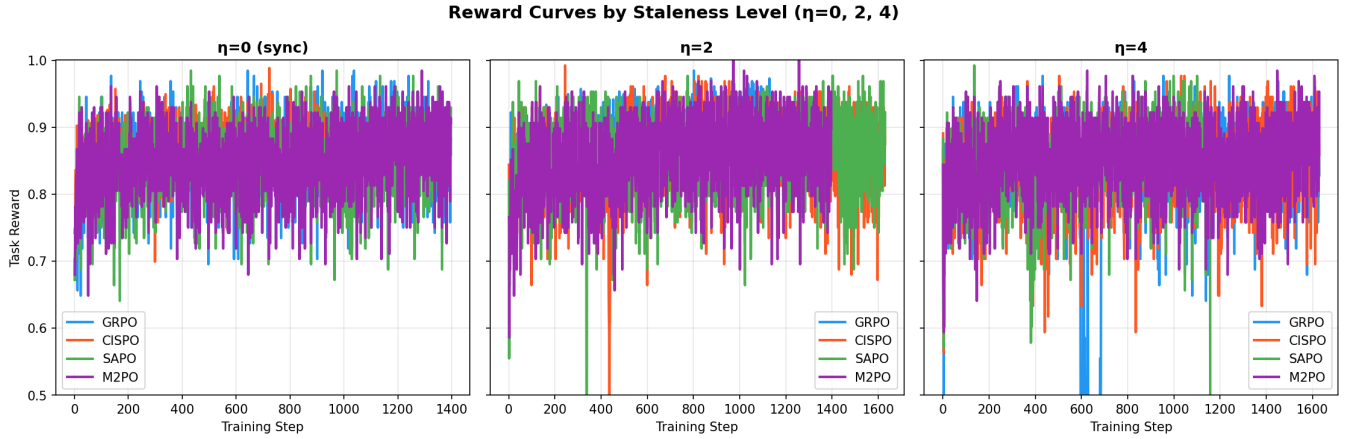


Figure 2: Training reward curves grouped by staleness level. At $\eta = 0$, all four algorithms occupy a similar reward band. At higher staleness, GRPO becomes noisier and trends lower, CISPO remains competitive, M2PO is comparatively smooth, and SAPO collapses in the high-staleness run.

setting, the most visible evaluation signal is instability rather than a consistent ordering of final scores.

GRPO’s evaluation reward under $\eta = 4$ shows a notable downward spike around step 650 before recovering, suggesting a transient instability that does not appear at lower staleness levels. CISPO’s evaluation curves are visually the tightest across staleness levels, as the three η conditions largely overlap throughout training. SAPO’s $\eta = 4$ evaluation curve collapses late in training, matching the near-zero final training reward. While evaluation does not separate all methods as strongly as final training reward, it supports the

qualitative conclusion that CISPO remains stable across the tested staleness range.

5.4 Team Member Contributions

- Sathya: Implemented CISPO, designed scripts to run AReaL experiments in cluster, and assisted with paper writing.
- Daniel: Researched AReaL and identified CISPO’s potential for asynchronous RL, ran experiments in cluster, and worked on paper.

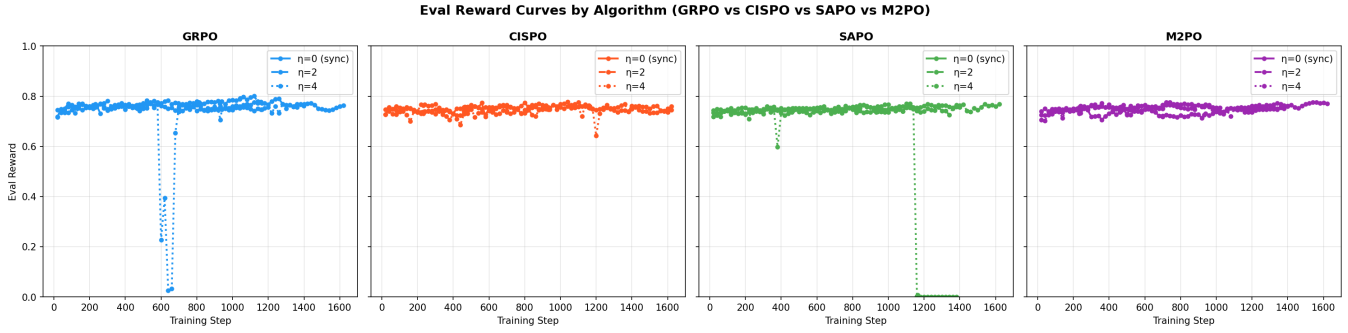


Figure 3: Evaluation reward over training, separated by algorithm. Each panel overlays $\eta \in \{0, 2, 4\}$ for a single algorithm. Evaluation curves are less separated than training reward, but GRPO and SAPO show visible instability at $\eta = 4$, while CISPO and M2PO remain tightly clustered across staleness levels.

- Ishaan: Assisted with AReaL experiment setup, performed results analysis, literature review, and contributed to project report writing and formatting.
- Parthav: Assisted with AReaL experiment setup, literature review, miscellaneous writing, and formatting.

6 Takeaways and Future Work

Our main findings are that CISPO achieves the highest final task reward at $\eta = 4$, M2PO maintains the smoothest training behavior across staleness levels, and GRPO’s task reward degrades approximately linearly with staleness despite competitive test-set evaluation. We discuss the likely mechanisms behind these results and their implications for algorithm selection in asynchronous RL systems.

6.1 CISPO in Asynchronous RL

CISPO was not originally designed for asynchronous training. Chen et al. [1] introduced it for synchronous reasoning-model training, where ratio clipping was found to suppress rare but informative reflective tokens. We hypothesized that the same mechanism could help under staleness: as stale rollouts push importance ratios away from one, CISPO preserves gradient signal from all valid tokens while bounding their influence. Our final training rewards support this hypothesis, with CISPO achieving the strongest result at $\eta = 4$ and remaining competitive at all tested staleness levels.

At the same time, the evaluation curves are less differentiated than the final training rewards. One likely reason is the benchmark: GSM8K is relatively easy compared to harder competition-math datasets. In RLVR settings, this can weaken the learning signal because advantages are normalized within response groups; if most rollouts in a group are correct, the advantage signal can become small. Future work should therefore repeat these experiments on harder datasets such as MATH [5], where algorithmic differences may appear more clearly in held-out evaluation.

6.2 M2PO and the High-Staleness Regime

M2PO is specifically designed for off-policy training, and the original paper [19] evaluates it at staleness levels substantially beyond the range we tested. In that regime, M2PO’s second-moment

region may outperform simpler importance-weight clipping once the policy gap becomes very large. Our compute constraints limited evaluation to $\eta \in \{0, 2, 4\}$, a range closer to AReaL’s recommended practical settings [8]. Within this moderate-staleness regime, M2PO trains smoothly but does not achieve the best final reward at $\eta = 4$. This suggests that CISPO may be a strong practical choice for realistic asynchronous workloads, while M2PO remains an important baseline for more extreme off-policy settings.

6.3 GRPO

GRPO’s roughly linear decline in reward with staleness is consistent with its design assumptions. Its ratio clipping is effective for on-policy or near-on-policy training, but provides no mechanism to handle the systematic ratio drift introduced by stale rollouts. As more tokens are clipped, gradient estimates can become noisier, resulting in greater variance in the reward curves. On GSM8K, this variance does not substantially affect test-set evaluation, but harder datasets may expose a larger gap between GRPO and methods designed to preserve more gradient signal under staleness.

6.4 SAPO and Incomplete Data

Several of our SAPO runs at high staleness failed to complete within the cluster’s wall-clock limit. The runs that did finish exhibit noisier learning curves than the other algorithms, but we cannot confidently claim this is due to staleness without further investigation. The original SAPO paper [4] reports stable off-policy performance, and we therefore attribute our partial results to a system-level constraint in our evaluation rather than to a deficiency of the algorithm. A more complete characterization of SAPO in this setup would require longer compute allocations as well as more detailed monitoring to diagnose the issues in our SAPO training runs.

6.5 Training speed

Our work does not evaluate end-to-end training speed. A valuable follow-up experiment would compare the speedup achieved by relaxing η against the resulting training instability. This would turn the current algorithmic comparison into a practical systems trade-off: for a given workload, how much stale data should the system permit before the loss in stability outweighs the throughput gain?

6.6 Challenges

The main practical challenge in this work was running AReaL under limited compute resources. AReaL has a large dependency stack and high GPU memory requirements, which made it difficult to run large sweeps on our setup. To make the experiments feasible, we reduced the model size, reduced the training batch size, and limited the number of experimental conditions. Future work would benefit from larger GPU allocations, repeated seeds, and broader hyperparameter sweeps.

7 Conclusion

We present a systematic comparison of four RL algorithms in AReaL under controlled staleness. By implementing CISPO and comparing it against GRPO, SAPO, and M2PO, we show that algorithm choice affects robustness to stale rollouts. CISPO achieves the strongest final training reward at high staleness in our experiments, M2PO provides smooth training but lower high-staleness final reward, GRPO degrades steadily, and SAPO is unstable in our $\eta = 4$ run. Our open-source CISPO implementation enables broader experimentation with this loss in AReaL. Future work should evaluate harder tasks, larger models, repeated seeds, and the throughput-stability trade-off induced by relaxing staleness constraints.

References

- [1] Aili Chen, Aonian Li, Bangwei Gong, Binyang Jiang, Bo Fei, Bo Yang, Boji Shan, Changqing Yu, Chao Wang, Cheng Zhu, et al. 2025. Minimax-m1: Scaling test-time compute efficiently with lightning attention. *arXiv preprint arXiv:2506.13585* (2025).
- [2] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168* (2021).
- [3] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, and Yi Wu. 2025. AReaL: A Large-Scale Asynchronous Reinforcement Learning System for Language Reasoning. doi:10.48550/ARXIV.2505.24298
- [4] Chang Gao, Chujie Zheng, Xiong-Hui Chen, Kai Dang, Shixuan Liu, Bowen Yu, An Yang, Shuai Bai, Jingren Zhou, and Junyang Lin. 2025. Soft adaptive policy optimization. *arXiv preprint arXiv:2511.20347* (2025).
- [5] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874* (2021).
- [6] Luke J Huang, Zhuoyang Zhang, Qinghao Hu, Shang Yang, and Song Han. 2026. Stable Asynchrony: Variance-Controlled Off-Policy RL for LLMs. *arXiv preprint arXiv:2602.17616* (2026).
- [7] Inclusion AI. 2026. AReaL: Asynchronous Reinforcement Learning Framework. <https://github.com/inclusionai/areal>. Accessed: May 2026.
- [8] Inclusion AI. 2026. AReaL Asynchronous RL Algorithm Documentation. <https://github.com/inclusionai/AReaL/blob/main/docs/en/algorithms/asyn.md>. Accessed: May 2026.
- [9] Haoyang Li, Sheng Lin, Fangcheng Fu, Yuming Zhou, Xiaodong Ji, Yanfeng Zhao, Lefeng Wang, Jie Jiang, and Bin Cui. 2026. Unleashing Efficient Asynchronous RL Post-Training via Staleness-Constrained Rollout Coordination. *arXiv preprint arXiv:2601.12784* (2026).
- [10] Michael Noukhovitch, Shengyi Huang, Sophie Xhonneux, Arian Hosseini, Rishabh Agarwal, and Aaron Courville. 2025. Asynchronous RLHF: Faster and More Efficient Off-Policy RL for Language Models. arXiv:2410.18252 [cs.LG] <https://arxiv.org/abs/2410.18252>
- [11] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 27730–27744. https://proceedings.neurips.cc/paper_files/paper/2022/file/b1efde53be364a73914f58805a001731-Paper-Conference.pdf
- [12] Alexandre Piché, Ehsan Kamaloo, Rafael Pardini, Xiaoyin Chen, and Dzmitry Bahdanau. 2025. Pipelinerl: Faster on-policy reinforcement learning for long sequence generation. *arXiv preprint arXiv:2509.19128* (2025).
- [13] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems* 36 (2023), 53728–53741.
- [14] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. doi:10.48550/ARXIV.1707.06347
- [15] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300* (2024).
- [16] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).
- [17] Ronald J Williams. 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning* 8, 3 (1992), 229–256.
- [18] Zhiheng Xi, Xin Guo, Yang Nan, Enyu Zhou, Junrui Shen, Wenxiang Chen, Jiaqi Liu, Jixuan Huang, Zhihao Zhang, Honglin Guo, et al. 2025. Bapo: Stabilizing off-policy reinforcement learning for llms via balanced policy optimization with adaptive clipping. *arXiv preprint arXiv:2510.18927* (2025).
- [19] Haizhong Zheng, Jiawei Zhao, and Beidi Chen. 2025. Prosperity before Collapse: How Far Can Off-Policy RL Reach with Stale Data on LLMs? doi:10.48550/ARXIV.2510.01161
- [20] Rui Zheng, Shihan Dou, Songyang Gao, Yuan Hua, Wei Shen, Binghai Wang, Yan Liu, Senjie Jin, Qin Liu, Yuhao Zhou, Limao Xiong, Lu Chen, Zhiheng Xi, Nuo Xu, Wenbin Lai, Minghao Zhu, Cheng Chang, Zhangyue Yin, Rongxiang Weng, Wensen Cheng, Haoran Huang, Tianxiang Sun, Hang Yan, Tao Gui, Qi Zhang, Xipeng Qiu, and Xuanjing Huang. 2023. Secrets of RLHF in Large Language Models Part I: PPO. doi:10.48550/ARXIV.2307.04964